

S-CTX — Plan-Determinism Contract + Gate: Decision Brief

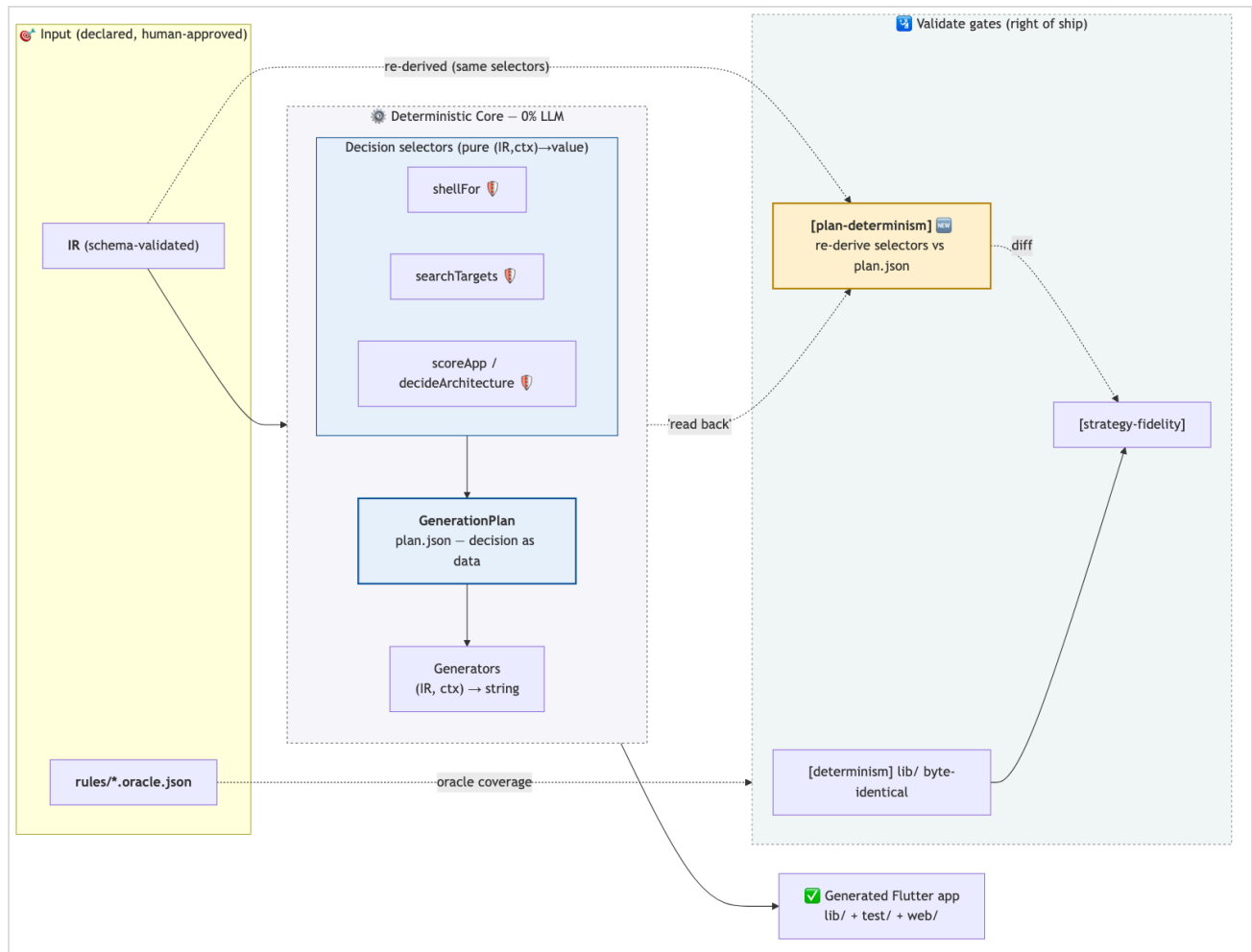
Owner decision brief · 2026-08-17 · Companion to SPIKE_PLAN.md §S-CTX (engineering spec)

Status: RECOMMENDED — implement as scoped.

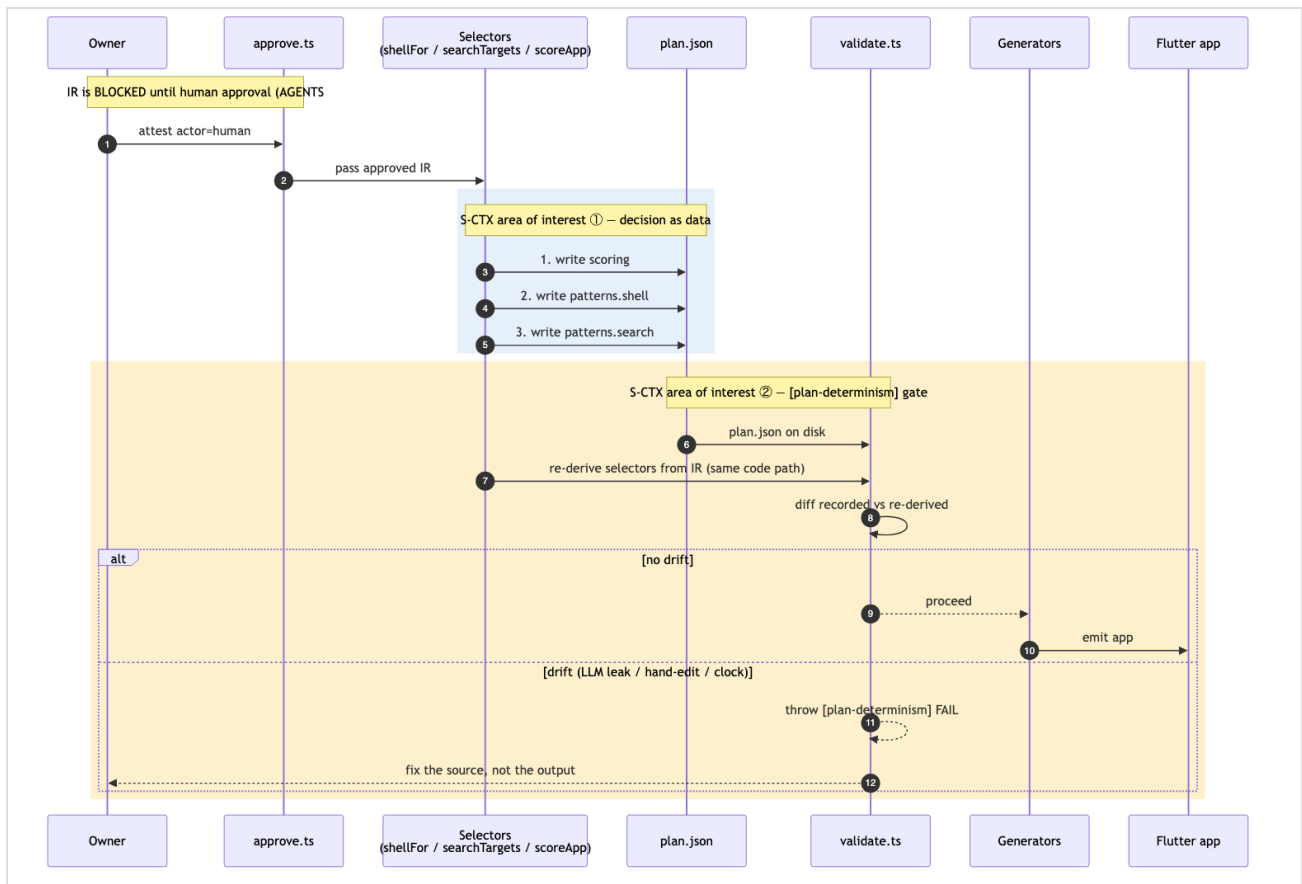
0. Diagrams — where S-CTX sits

■ amber = area of interest ①/② this brief decides on ■ blue = "decision as data" (plan selectors) ▭ dashed = gate/validate grouping

Pipeline (IR → selectors → plan → generators → gates)



Sequence — plan written, then [plan-determinism] re-derives + diffs



1. Executive summary

The generator's plan (`plan.json`) is already 100% derived from the IR through a handful of pure selector functions (`shellFor` , `searchTargets` , `scoreApp`). That is a *fact*, but nothing **proves** it stays true — a future accidental LLM call, wall-clock read, or hand-edit inside any plan-field helper would silently make the plan non-deterministic, and no gate catches it. S-CTX is a small, zero-generator-change slice that:

1. **Writes down** exactly what composes the plan, field by field, with each derivation cited (a `DETERMINISM_CONTRACT.md`).
2. **Adds one validate gate** (`[plan-determinism]`) that re-derives the plan decisions from the IR and diffs against the `plan.json` on disk.

Effort ~S (1 slice). Impact: hardens the trust boundary (AGENTS #3: "deterministic core is 0% LLM") into a *checked* invariant. No generated code, IR, or schema changes.

2. Context

- Non-negotiable #3: deterministic core is **0% LLM** — generators are pure (`IR` , `ctx`) → `string` . The plan (`plan.ts`) and gen context (`gen_context.ts`) are the "decision as data" layer.
- Grilling round 1 surfaced **C1** ("ctx is undefined; determinism invariant is a tautology") and **C15** ("an LLM-authored plan recurses nondeterminism"). Both answered with "it's already

true, verified by code reading" — but no *standing check*.

- ChatGPT round-2 hardening: "IR-derived" must be **transitive** — every helper a plan-field derivation calls must itself be pure (no wall clock / filesystem / network / randomness / mutable process state).
- Roadmap gates S-CTX first (cheap; later spikes P3/P4/P5-D2 cite `plan.json`). P2 closed; M4a just shipped — this is the natural next slice.

3. Why it matters (impact)

Impact axis	With S-CTX	Without
Trust boundary (AGENTS #3)	"0% LLM" is a <i>checked</i> invariant	unenforced claim; a leak ships silently
Determinism guarantee	plan re-proven from source every validate	plan can drift from IR without notice
Later slices (P3/P4/P5-D2)	build on a guaranteed-stable <code>plan.json</code>	inherit a possibly-stale plan
Audit / explainability	plan provenance documented field-by-field	"where did this decision come from?" is tribal knowledge
Risk	low — additive gate, negative-control proven	structural risk grows with each pattern slice

4. Options (with priority + impact)

#	Option	Priority	Effort	Impact	Verdict
A	Full S-CTX: contract doc + [plan-determinism] gate + negative control	P1	S	High — hardens core trust boundary; unblocks roadmap	RECOMMENDED
B	Gate only, no contract doc	P2	XS	Medium — catches drift but leaves grill C1 half-answered	acceptable fallback
C	Contract doc only, no gate	P3	XS	Low — documentation without enforcement	not recommended
D	Do nothing (rely on "it's true" code-reading)	—	0	None — unproven claim; risk grows per slice	rejected
E	Extend scope: prove generator purity transitively (scan imports)	P3	M	High but larger blast radius; violates small-slice rule	DEFER

5. Findings (grounded, from code)

- `GenContext ( gen_context.ts:18-24 ) = pkg , symbols , ir , sm , search` — all computed from IR + arch layer. No LLM, clock, or FS.
- `GenerationPlan ( plan.ts:26-40 )`: only `scoring` + `patterns.{shell,search}` are decision-bearing; both come from pure selectors:
 - `scoring ← decideArchitecture(ir) ( arch.ts:42 ) → scoreApp ( scoring.ts:108 )` — pure.
 - `patterns.shell ← shellFor(features, mergedScreens) ( composition.ts:111 )` — pure, feature order preserved.
 - `patterns.search ← searchTargets(ir) ( composition.ts:173 )` re-keyed by `screenPath ( index.ts:723 )` — pure.
- `searchCheck ( validate.ts:328 )` is the existing precedent — it already re-derives search decisions from the IR and diffs `plan.json`. The new gate generalizes this to `scoring` + `patterns.shell` + `patterns.search`.
- Determinism gate (`validate.ts:747`) already regenerates + diffs `lib/` ; the new gate is additive alongside it (`plan.json` vs IR).
- No current app declares a matching `stateMachines` surface (M4a finding) — strategy strings are stable; the gate is cheap to add now.

6. Decision

GO — Option A. Implement as one small slice:

1. `research/DETERMINISM_CONTRACT.md` — field-by-field derivation map + transitive-purity invariant.
2. `validate.ts` — `[plan-determinism]` gate re-deriving `scoring` + `patterns.*` from IR, diffed against `plan.json` on disk.
3. Negative control — hand-edit `plan.json` `patterns.shell` → gate FAILs (proves non-vacuous).
4. Full verify + commit + `HANDOFF/CODE_CATALOGUE` update.

Acceptance: passes on all 4 apps + all samples; negative control fails; typecheck clean; no generator changes.